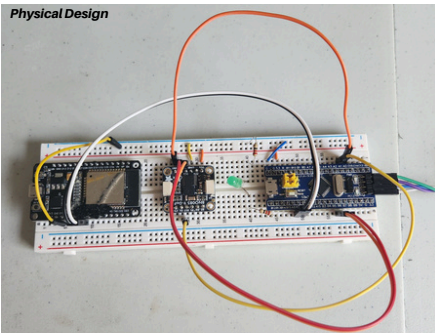


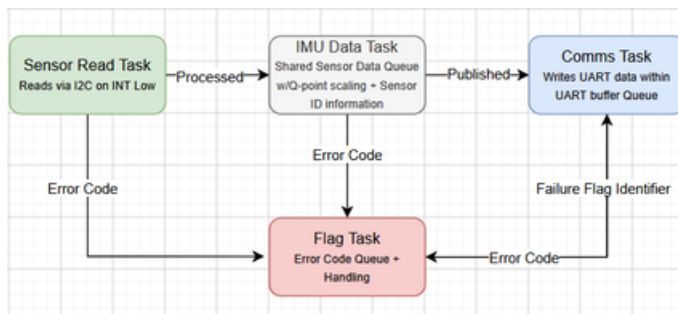
WIRELESS SENSOR CONTROL HUB



Situation

- Needed a **real-time embedded platform** to continuously acquire, process and wirelessly transmit multi-axis inertial and magnetic sensor data
- Required a **memory-constrained RTOS implementation** operating within the STM32 Blue Pill's 20KB RAM while maintaining concurrent sensor acquisition and communication tasks

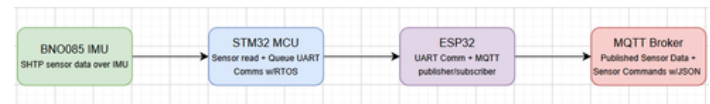
STM32 RTOS Architecture



Results

- Successfully **streamed calibrated real-time data** from all three BNO085 sensors simultaneously
- Resolved multiple low-level debugging challenges including **stack overflow diagnosis** under memory constraints, SHTP channel flooding from concurrent sensor streams and **FreeRTOS heap exhaustion** from task creation
- Achieved **wireless firmware deployment** capability via OTA updates without physical device access

System Pipeline



Method

- Architected a FreeRTOS multi-task pipeline on the STM32 with dedicated tasks for **IMU**, **UART**, and **status indication**, using message queues for communication under a 20KB RAM constraint
- Manually implemented **SHTP driver over I2C**, handling boot sequence, channel setup, and sensor setup enabling **concurrent streaming** from all three BNO085 sensors
- Bridged structured sensor data over **UART** to an ESP32 **MQTT** publisher supporting **OTA firmware** updates

MQTT Broker Live Output

2026-07-18 19:19:06:538

```
Topic: esp32/unit01/data  QoS: 0
{
  "ID": 3,
  "X": 34.44,
  "Y": 33.38,
  "Z": -35.13
}
```

2026-07-18 19:19:06:539

```
Topic: esp32/unit01/data  QoS: 0
{
  "ID": 1,
  "X": 0.15,
  "Y": -0.19,
  "Z": 9.65
}
```

2026-07-18 19:19:06:698

Features

- FreeRTOS multi-task **architecture with message queues** enabling concurrent sensor acquisition and **UART transmission** within a **20KB RAM constraint**
- Manual **SHTP protocol** implementation over **I2C handling** advertisement parsing, Set Feature commands and multi-sensor data report parsing **without external libraries**
- Real-time accelerometer, gyroscope and magnetometer fusion w/Q-point scaling
- **MQTT wireless telemetry pipeline** via ESP32 bridge with OTA firmware update support

MICROMOUSE - UGA IEEE

International MicroMouse Competition



Navigation Feedback Loop Code

```
119 //Get information from all 12 sensors
120 for (int i = 0; i < 12; i++)
121 {
122     //If sensors are pre-set configured to follow M0_001 config
123     M0_001 dir = 1; //If sensors are pre-set configured to follow M0_001 config
124
125     //wall exists in direction
126     if (downcast<float>(float) ADC_WAL1[i])
127     {
128         //From direction of sensor, determine direction
129         ABS_dir = solve_direction(direction: dir, clipped_heading: heading);
130
131         //update internal representations
132         updateWall(wall_location: true_dir, maze_grid: rep.wall, x: rep.x, y: rep.y);
133         updateLocation(wall_dir: true_dir, grid: rep.grid, heading: x: rep.x, y: rep.y);
134         //walls are defaulted to not exist, so no further code is necessary
135     }
136
137     //Find where the bot should move relatively
138     M0_001 path = findPath(grid: rep.grid);
139
140     //Travel in said direction
141     travelPath(dir: path, goal_index: MOTOR_PINS);
142
143     //update location accordingly
144     updateLocation(dir: path, x: rep.x, y: rep.y);
145
146     M0_001 delay: 100;
147 }
```

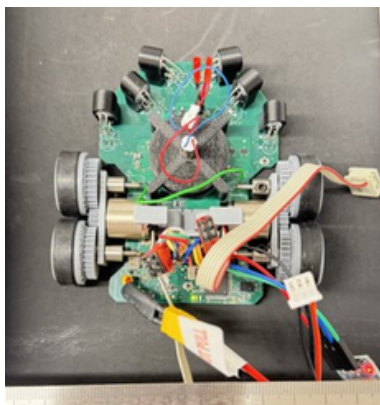
Method

- Utilized the **STM32 MCU** as the target hardware to develop all firmware
- Managed concurrent **I2C**, **UART PWM**, and **ADC peripherals** on STM32 using CubeMX and **HAL** drivers for IMU heading, Bluetooth telemetry, motor control, and wall detection
- Integrated all features into a **floodfill navigation firmware** in **embedded C**

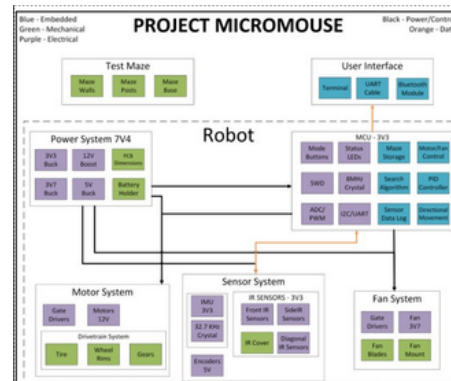
Situation

- Needed autonomous maze-solving firmware for a 16x16 competition maze with **real-time** wall detection and **dynamic path calculation**
- Required a navigation firmware that could run efficiently within **limited RAM** without blocking time-critical **peripheral communication**

Fully developed MicroMouse robot



Sensor Communication Diagram



Results

- Demonstrated **full maze solve** on test runs before competition, with firmware reliably managing all peripherals simultaneously under real conditions
- Implemented **live Bluetooth telemetry** to validate firmware state during development, confirming correct peripheral timing and navigation decisions in real time

Features

- Multi-peripheral firmware architecture** coordinating ADC, I2C, UART, and PWM simultaneously on a single STM32
- Real-time floodfill execution** within RAM and CPU constraints leveraging **DMA**-offloaded sensor reads, allowing for non-blocking simultaneous peripheral communication
- Wireless debug pipeline** through UART/Bluetooth for **live firmware state monitoring** during testing